

SQL Practice Worksheet

DDL & DML — Medium Level — Library Management System

Instructions: All questions revolve around a single scenario — a Library Management System. Work through the questions in order, since later questions build on the table created earlier. Write your SQL queries for each question. Assume MySQL syntax unless stated otherwise.

Q1. Creating the Table (Constraints)

Create a table named **Books** with the following columns:

- **book_id** – integer, Primary Key, auto-increment
- **title** – text (max 100 characters), cannot be left empty
- **isbn** – text (max 20 characters), must be unique for every book
- **price** – decimal value with 2 digits after the decimal point
- **published_date** – a date value
- **in_stock** – integer, should default to 1 if no value is given

Hint: Use PRIMARY KEY, AUTO_INCREMENT, NOT NULL, UNIQUE and DEFAULT together in one CREATE TABLE statement.

Q2. Inserting Data

Insert **5 records** into the *Books* table with different titles, ISBNs, prices, and published dates. Leave the *book_id* out of your INSERT statement (let it auto-increment), and let one record use the default value of *in_stock*.

Hint: When a column has AUTO_INCREMENT or DEFAULT, you can omit it from the column list of the INSERT statement.

Q3. Renaming a Column

The library staff feel the column name *title* is confusing. Rename the column **title** to **book_title** in the *Books* table, without affecting any data already stored in it.

Hint: ALTER TABLE ... RENAME COLUMN ... TO ... ;

Q4. Changing a Column's Data Type

Change the data type of the **price** column from DECIMAL(8,2) to **FLOAT**, so very large book prices can be stored without precision errors.

Hint: ALTER TABLE ... MODIFY COLUMN ... (MySQL) or ALTER COLUMN ... TYPE ... (PostgreSQL).

Q5. Adding a New Column with a Constraint

Add a new column **author_name** (text, max 50 characters) to the *Books* table. The column must not allow NULL values, and should have the default value 'Unknown' for existing rows.

Hint: ALTER TABLE ... ADD COLUMN author_name VARCHAR(50) NOT NULL DEFAULT 'Unknown';

Q6. Updating Existing Data

Give a **10% discount** on the price of every book that was published before the year 2020. Update the *price* column accordingly using a single UPDATE statement.

Hint: `UPDATE ... SET price = price * 0.9 WHERE published_date < '2020-01-01';`

Q7. Deleting Specific Records

Remove all books from the *Books* table whose **in_stock** value is 0 (meaning the book is currently out of stock and no longer tracked).

Hint: `DELETE FROM Books WHERE in_stock = 0;`

Q8. Resetting the Auto-Increment Value

After cleaning up old records, the librarian wants new books to start getting **book_id** values from **101** onward instead of continuing from where it left off. Write the statement to reset the auto-increment counter of the *Books* table to 101.

Hint (MySQL): `ALTER TABLE Books AUTO_INCREMENT = 101;`

Q9. Conditional SELECT with Pattern Matching

Display the *book_title*, *price* and *published_date* of all books whose title contains the word 'History', and whose price is greater than 300. Sort the result by price in descending order.

Hint: Combine `WHERE`, `LIKE '%History%'`, `AND`, and `ORDER BY ... DESC`.

Q10. Working with a Second Table (Joins-ready setup)

Create another table named **Members** with columns: **member_id** (integer, Primary Key, auto-increment), **member_name** (text, not null), and **join_date** (date, not null, defaults to today's date). Then write a query to display all members who joined in the year 2024, ordered by member_name alphabetically.

Hint: Use `CURRENT_DATE` (or `CURDATE()` in MySQL) as the default for *join_date*, and `YEAR(join_date) = 2024` in the `WHERE` clause.

Tip: Try solving each question on paper or in an SQL editor (MySQL Workbench / SQLite / phpMyAdmin) and verify the output by running a `SELECT *` after every DDL/DML statement.